OS & SP

Name : D. Teja Sadwika **Roll No :** 2520030536

SKILL – 24

1.Skill24.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>

void run_demo(const char *name, int seconds)
{
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return;
```

```
    }

    if (pid == 0) {
        printf("[START] %s | PID=%d\n",
               name, getpid());
        fflush(stdout);

        sleep(seconds);

        printf("[DONE] %s | PID=%d\n",
               name, getpid());
        fflush(stdout);

        exit(0);
    }

    waitpid(pid, NULL, 0);
}
```

```
int main()
{
    printf("=====================================\n");
    printf(" OSSP PROGRAM 24 - FINAL DEMO\n");
    printf("=====================================\n\n");

    /* Demo 1: Process execution */
    printf("Demo 1: Process execution\n");
    run_demo("Process Demo", 1);

    /* Demo 2: Pipeline-style jobs */
    printf("\nDemo 2: Pipeline-style jobs\n");
    fflush(stdout);

    pid_t p1 = fork();

    if (p1 < 0) {
```

```
    perror("fork");
    return 1;
}

if (p1 == 0) {
    printf("Pipeline Job 1 | PID=%d\n", getpid());
    fflush(stdout);
    sleep(1);
    exit(0);
}

pid_t p2 = fork();

if (p2 < 0) {
    perror("fork");
    waitpid(p1, NULL, 0);
    return 1;
}
```

```
if (p2 == 0) {
    printf("Pipeline Job 2 | PID=%d\n", getpid());
    fflush(stdout);
    sleep(2);
    exit(0);
}

waitpid(p1, NULL, 0);
waitpid(p2, NULL, 0);

printf("Pipeline jobs completed.\n");

/* Demo 3: Signal handling */
printf("\nDemo 3: Signal handling\n");
fflush(stdout);

pid_t child = fork();
```

```
if (child < 0) {
    perror("fork");
    return 1;
}

if (child == 0) {
    printf("Signal demo process started | PID=%d\n",
          getpid());
    fflush(stdout);

    sleep(5);

    printf("Signal demo completed.\n");
    exit(0);
}

sleep(1);
```

```
    printf("Sending SIGTERM to child...\n");
    fflush(stdout);

    if (kill(child, SIGTERM) < 0) {
        perror("kill");
    }

    waitpid(child, NULL, 0);

    printf("Signal demonstration completed.\n");

    printf("\n====================================\n");
    printf(" FINAL PROJECT REVIEW COMPLETE\n");
    printf("====================================\n");

    return 0;
}
```

OUTPUT :

```
sadwika@DESKTOP-8CQAO1L:~$ nano skill24.c
sadwika@DESKTOP-8CQAO1L:~$ gcc skill24.c -o skill24
sadwika@DESKTOP-8CQAO1L:~$ ./skill24
====================================
 OSSP PROGRAM 24 - FINAL DEMO
====================================

Demo 1: Process execution
[START] Process Demo | PID=3007
[DONE] Process Demo | PID=3007

Demo 2: Pipeline-style jobs
Pipeline Job 1 | PID=3008
Pipeline Job 2 | PID=3009
Pipeline jobs completed.

Demo 3: Signal handling
Signal demo process started | PID=3010
Sending SIGTERM to child...
Signal demonstration completed.

====================================
 FINAL PROJECT REVIEW COMPLETE
====================================
sadwika@DESKTOP-8CQAO1L:~$
```